



STEP 1: Import & explore data

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
```

```
In [2]: data = pd.read_csv(r'C:\Users\li\Desktop\Arbor Academy Project\ecommerce_data.csv')
```

```
In [3]: display(data.copy())
```

	OrderID	ProductPrice	Quantity	CustomerAge	ProductReviews	ProductRa
0	1	839.68	1.0	44.184047	18.0	
1	2	47.23	1.0	11.608463	11.0	
2	3	62.95	4.0	18.834189	26.0	
3	4	80.46	1.0	40.147378	NaN	
4	5	45.32	3.0	44.344160	19.0	
...	...	...	...	...	...	...
2005	6	45.32	1.0	44.623125	19.0	
2006	7	81.58	2.0	23.194456	22.0	
2007	8	65.35	5.0	20.326015	19.0	
2008	9	40.61	4.0	27.400477	20.0	
2009	10	NaN	3.0	38.598091	22.0	

2010 rows × 14 columns

Basic Info

Checking data types

```
In [4]: display(data.head())
display(data.tail())
print(data.shape)
print(data.info)
display(data.describe())
```

	OrderID	ProductPrice	Quantity	CustomerAge	ProductReviews	ProductRating
0	1	839.68	1.0	44.184047	18.0	2.58
1	2	47.23	1.0	11.608463	11.0	4.41
2	3	62.95	4.0	18.834189	26.0	4.02
3	4	80.46	1.0	40.147378	NaN	3.07
4	5	45.32	3.0	44.344160	19.0	3.29

	OrderID	ProductPrice	Quantity	CustomerAge	ProductReviews	ProductRa
2005	6	45.32	1.0	44.623125	19.0	
2006	7	81.58	2.0	23.194456	22.0	
2007	8	65.35	5.0	20.326015	19.0	
2008	9	40.61	4.0	27.400477	20.0	
2009	10	NaN	3.0	38.598091	22.0	

(2010, 14)

<bound method DataFrame.info of

	Age	ProductReviews \		OrderID	ProductPrice	Quantity	Customer
0		1	839.68	1.0	44.184047	18.0	
1		2	47.23	1.0	11.608463	11.0	
2		3	62.95	4.0	18.834189	26.0	
3		4	80.46	1.0	40.147378	NaN	
4		5	45.32	3.0	44.344160	19.0	
...	...	...	...	...	...	...	...
2005		6	45.32	1.0	44.623125	19.0	
2006		7	81.58	2.0	23.194456	22.0	
2007		8	65.35	5.0	20.326015	19.0	
2008		9	40.61	4.0	27.400477	20.0	
2009		10	NaN	3.0	38.598091	22.0	

	ProductRating	DiscountPercentage	CustomerTenure	ProductCategory \
0	2.58	45.44	15.84	Clothing
1	4.41	47.51	7.92	Sports
2	4.02	13.46	30.24	Home
3	3.07	37.71	52.10	Clothing
4	3.29	11.94	0.01	Sports
...	...	...	...	...
2005	1.52	36.88	27.94	Electronics
2006	4.36	39.87	34.96	Clothing
2007	4.15	12.36	5.66	Sports
2008	3.63	35.79	16.14	Sports
2009	2.86	40.61	2.24	Books

	DeliveryType	PaymentMethod	CustomerLocation	ReturnStatus	BrowsingTime
0	Express	PayPal	Central	Yes	212.17
1	Standard	Debit Card	Central	Yes	96.08
2	Express	Debit Card	North	No	115.24
3	Overnight	UPI	North	Yes	136.18
4	Express	PayPal	Central	Yes	17.15
...	...	...	...	...	...
2005	Standard	Credit Card	East	Yes	142.69
2006	Express	Credit Card	North	Yes	51.85
2007	Standard	Credit Card	Central	No	124.08
2008	Standard	PayPal	South	No	109.82
2009	Overnight	Debit Card	West	Yes	270.85

[2010 rows x 14 columns]>

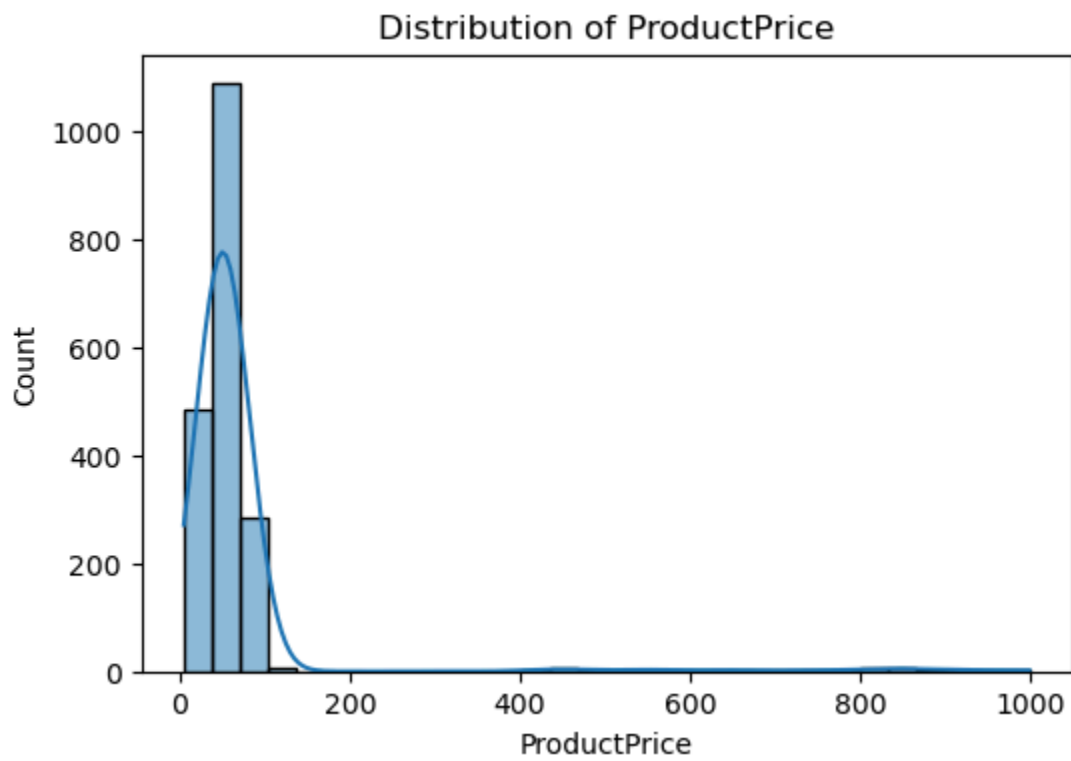
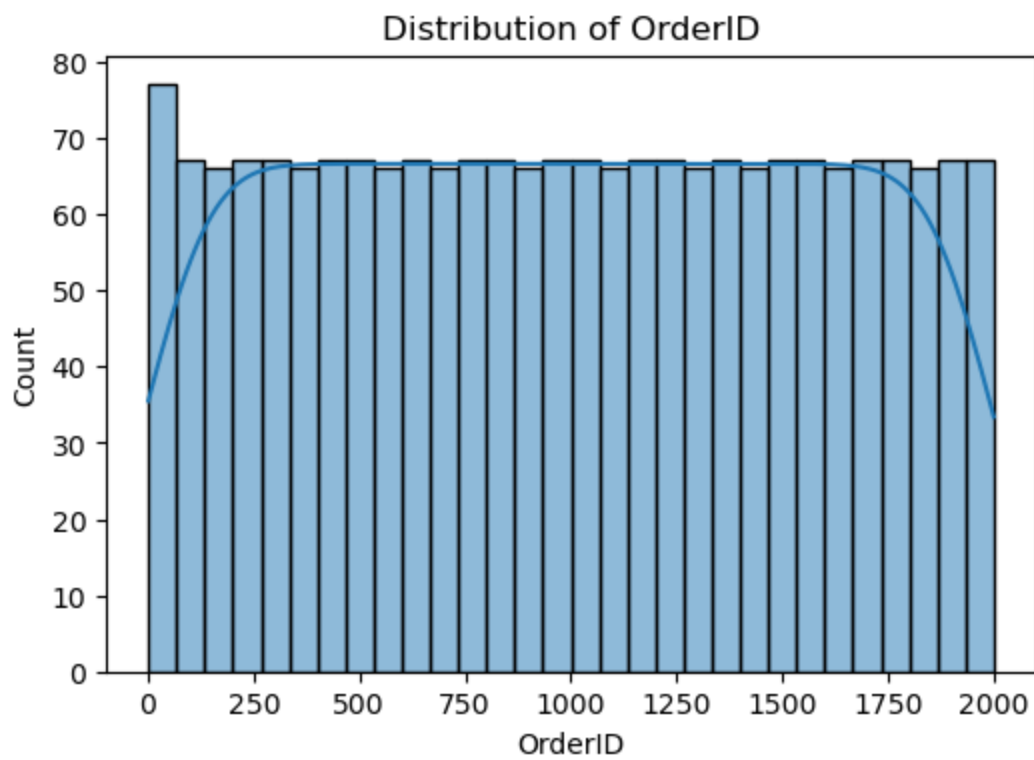
	OrderID	ProductPrice	Quantity	CustomerAge	ProductReviews	Pr
count	2010.000000	1912.000000	1911.000000	1911.000000	1909.000000	
mean	995.549751	68.603279	4.897436	36.304163	19.991095	
std	580.295996	111.933226	17.690710	19.578622	4.456689	
min	1.000000	5.000000	0.000000	-23.836004	7.000000	
25%	493.250000	37.970000	2.000000	25.268992	17.000000	
50%	995.500000	51.400000	3.000000	35.491605	20.000000	
75%	1497.750000	65.327500	4.000000	45.021130	23.000000	
max	2000.000000	998.750000	200.000000	200.000000	37.000000	

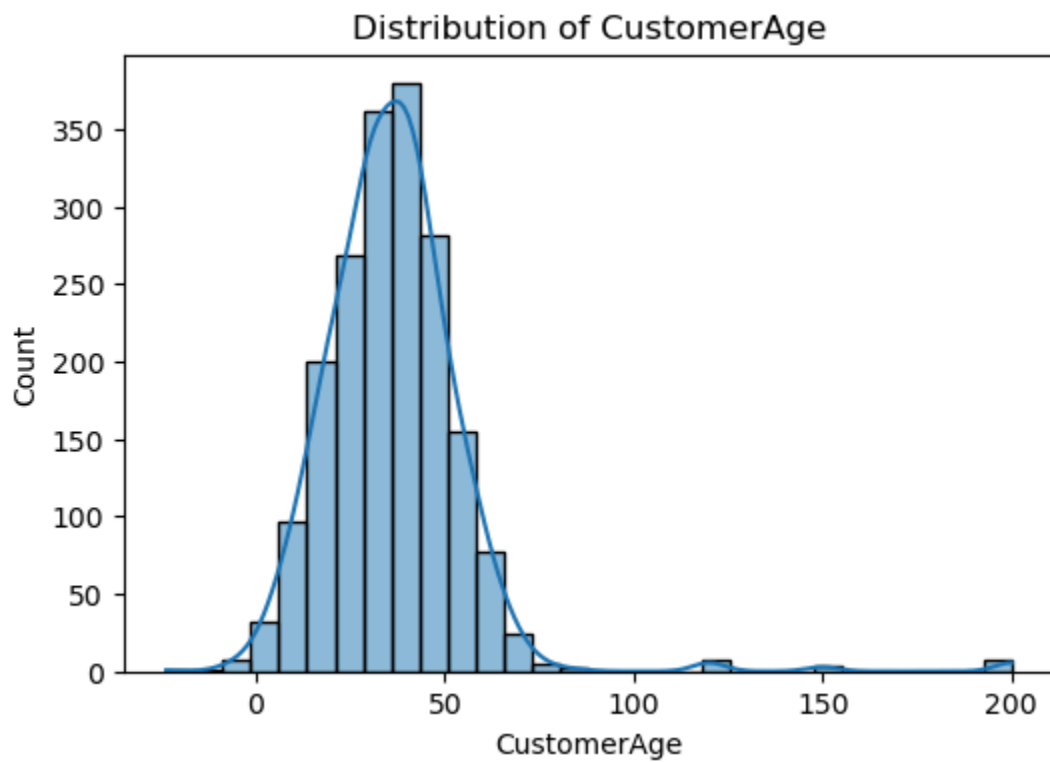
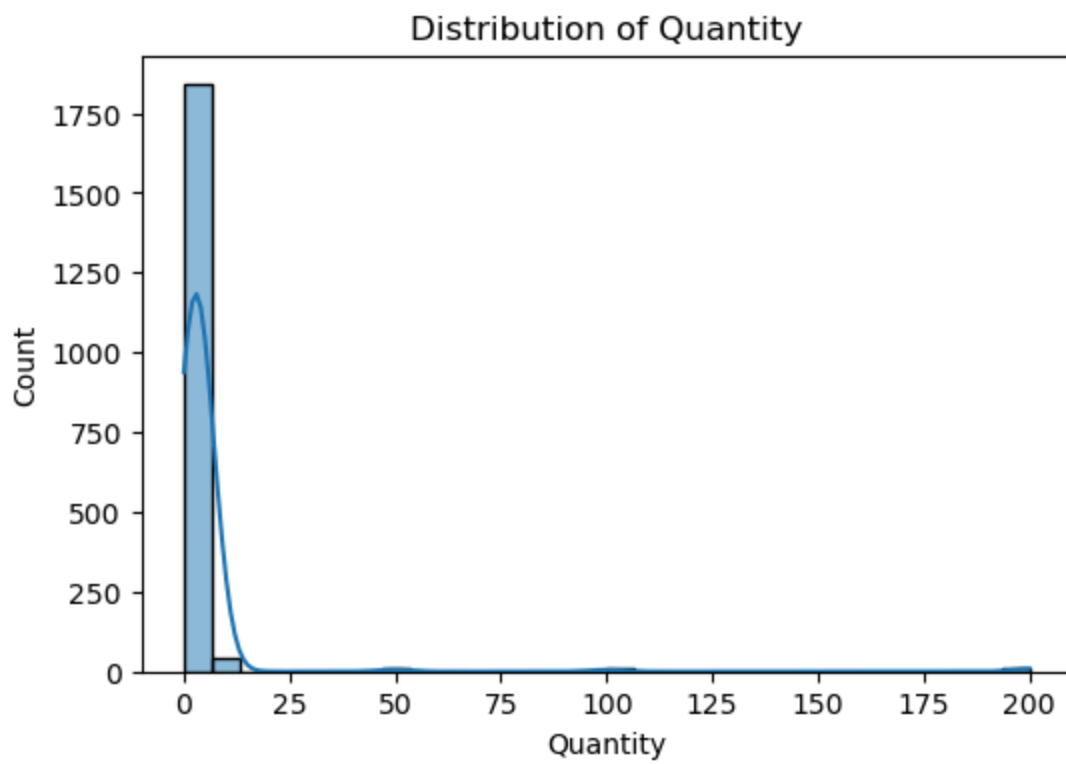
Visualize Distribution

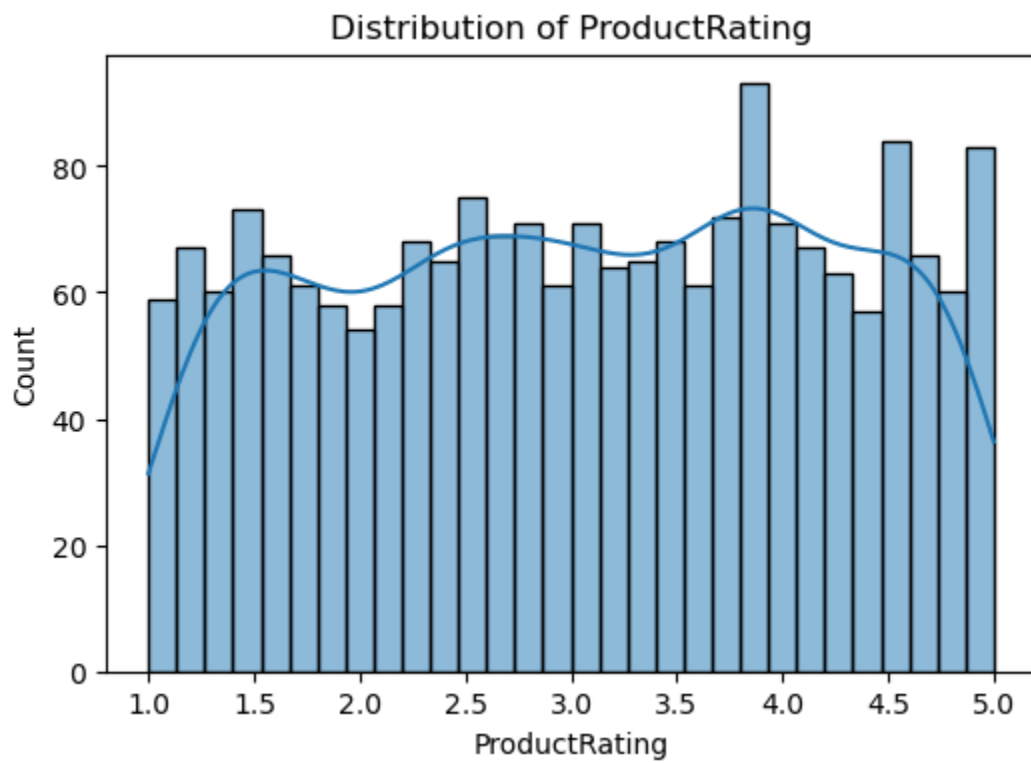
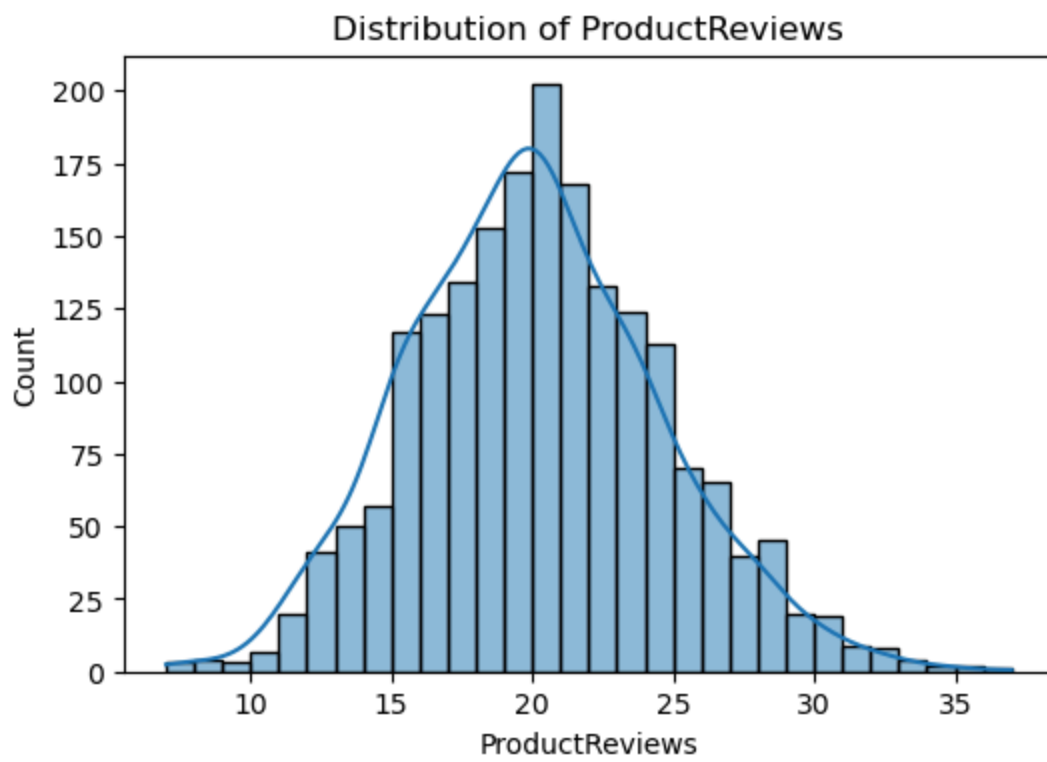
In [5]: `print(data.dtypes)`

```
OrderID          int64
ProductPrice     float64
Quantity         float64
CustomerAge      float64
ProductReviews   float64
ProductRating    float64
DiscountPercentage float64
CustomerTenure   float64
ProductCategory  object
DeliveryType     object
PaymentMethod    object
CustomerLocation object
ReturnStatus     object
BrowsingTime     float64
dtype: object
```

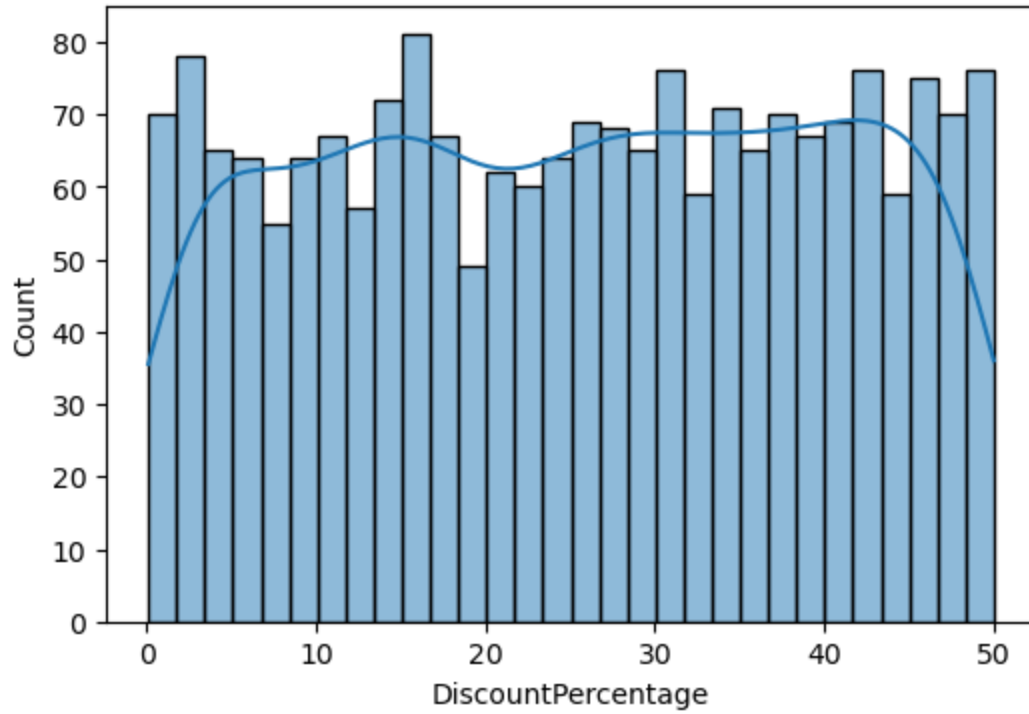
In [6]: `numeric_cols = data.select_dtypes(include=np.number).columns`
`for col in numeric_cols:`
 `plt.figure(figsize=(6,4))`
 `sns.histplot(data[col], kde=True, bins=30)`
 `plt.title(f'Distribution of {col}')`
 `plt.show()`



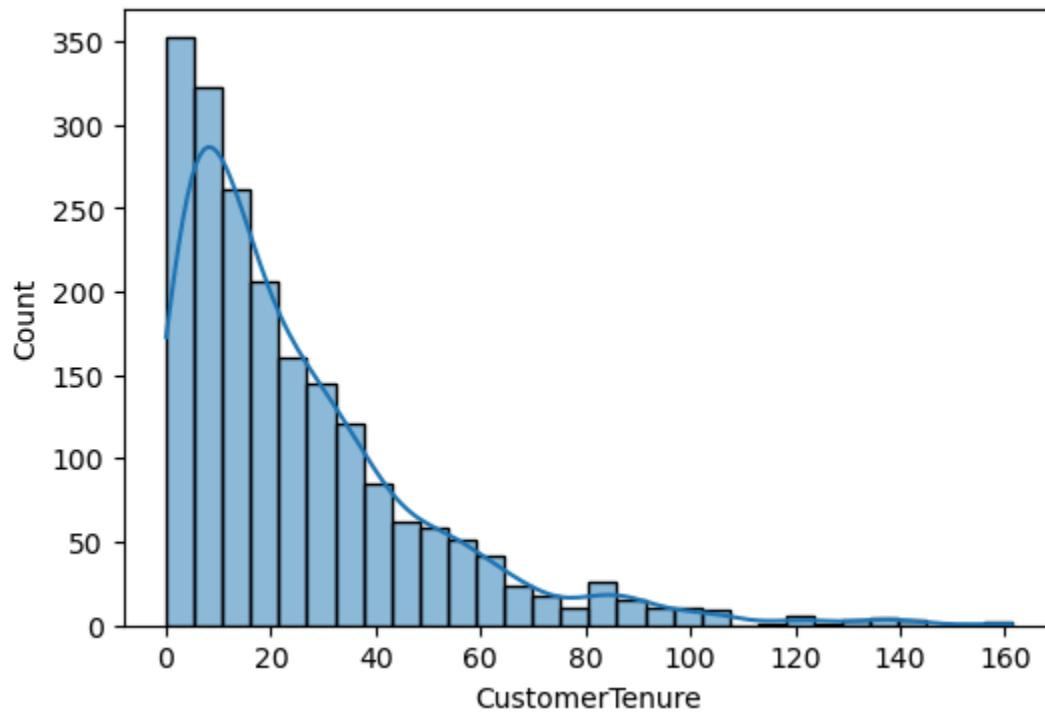


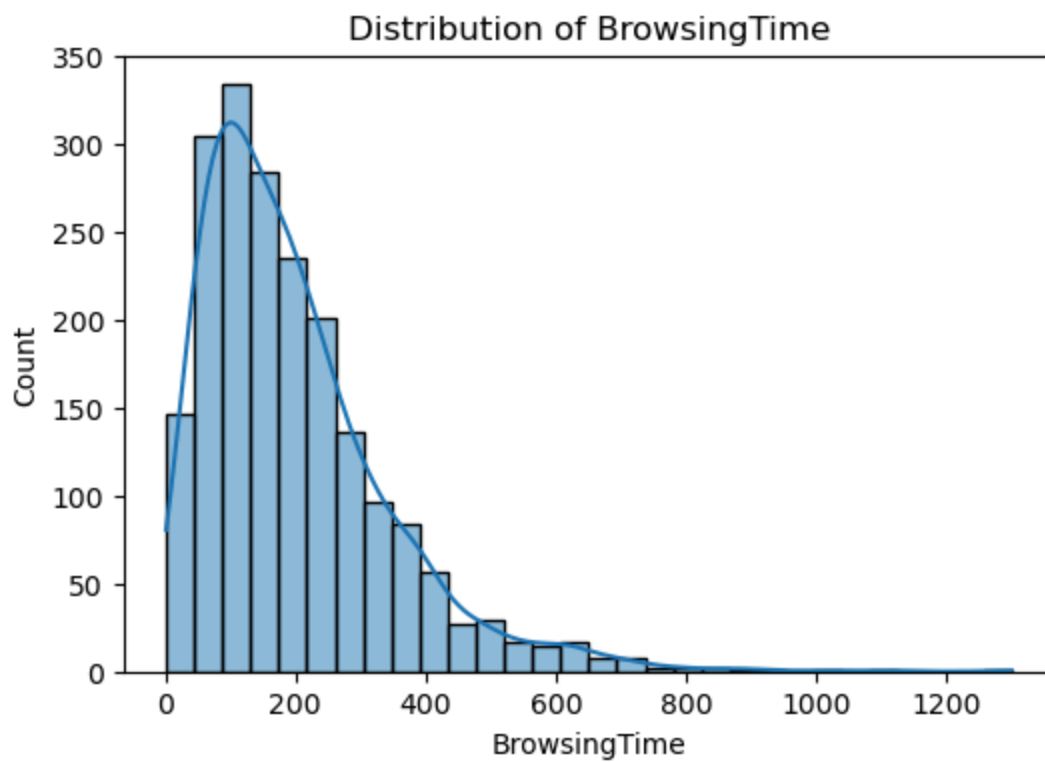


Distribution of DiscountPercentage



Distribution of CustomerTenure





```
In [7]: # Identify duplicate rows
duplicates = data[data.duplicated()]
```

```
In [8]: # Remove exact duplicates
data = data.drop_duplicates()
```

```
In [9]: # Log after removal
print("Duplicates removed. New shape:", data.shape)
```

Duplicates removed. New shape: (2000, 14)

```
In [10]: data
```

```
Out[10]:
```

	OrderID	ProductPrice	Quantity	CustomerAge	ProductReviews	ProductI
0	1	839.68	1.0	44.184047	18.0	
1	2	47.23	1.0	11.608463	11.0	
2	3	62.95	4.0	18.834189	26.0	
3	4	80.46	1.0	40.147378	NaN	
4	5	45.32	3.0	44.344160	19.0	
...	...	...	...	...	...	...
1995	1996	71.40	5.0	38.211665	18.0	
1996	1997	49.47	2.0	42.336958	17.0	
1997	1998	32.36	5.0	35.827560	21.0	
1998	1999	46.74	1.0	34.990873	18.0	
1999	2000	35.10	2.0	52.949407	15.0	

2000 rows × 14 columns

STEP 3: Handle Missing Values

```
In [11]: display(data.isnull().sum())
```

```
OrderID                0
ProductPrice           97
Quantity              99
CustomerAge            99
ProductReviews        100
ProductRating          0
DiscountPercentage     0
CustomerTenure         0
ProductCategory        0
DeliveryType           0
PaymentMethod          0
CustomerLocation       0
ReturnStatus           0
BrowsingTime           0
dtype: int64
```

```
In [12]: # Impute numerical columns with median
data['ProductPrice'] = data['ProductPrice'].fillna(data['ProductPrice'].median())
data['Quantity'] = data['Quantity'].fillna(data['Quantity'].median())
data['CustomerAge'] = data['CustomerAge'].fillna(data['CustomerAge'].median())
data['ProductReviews'] = data['ProductReviews'].fillna(data['ProductReviews'].median())
```

```
In [13]: # Verify missing values handled
print(data[['ProductPrice', 'Quantity', 'CustomerAge', 'ProductReviews']].isnull().sum())
```

```
ProductPrice      0
Quantity          0
CustomerAge       0
ProductReviews    0
dtype: int64
```

Detect & Treat Outlier

```
In [14]: # Function to cap outliers using IQR for multiple columns
def cap_outliers(data, columns):
    for col in columns:
        Q1, Q3 = np.percentile(data[col], [25, 75])
        IQR = Q3 - Q1
        lower, upper = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
        outliers = ((data[col] < lower) | (data[col] > upper)).sum()

        # Cap outliers
        data[col] = np.clip(data[col], lower, upper)
        print(f"{col}: {outliers} outliers capped.")

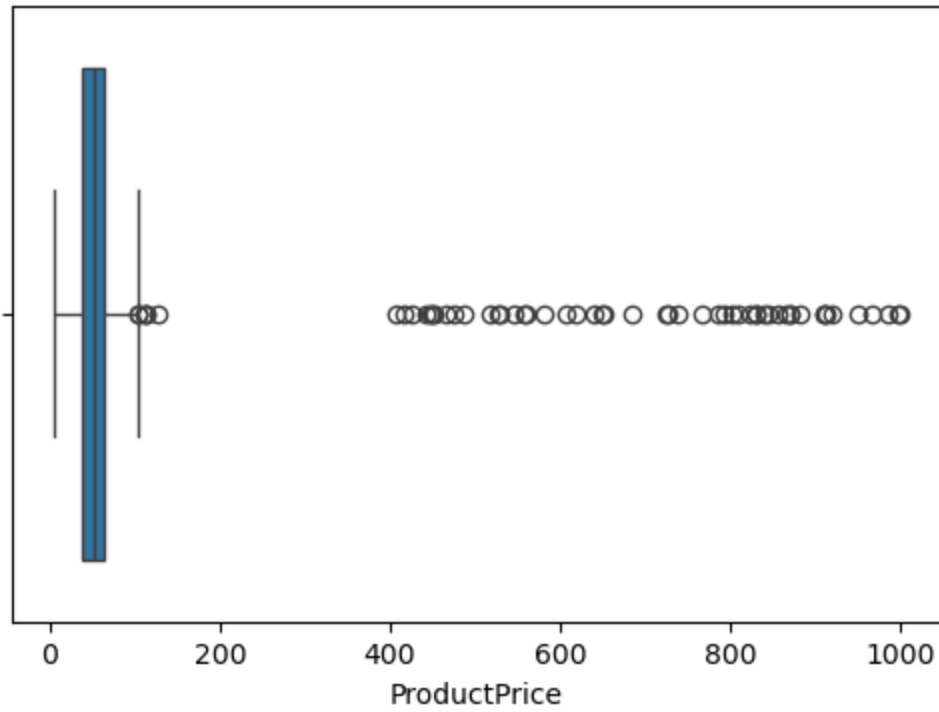
    print("\n Outlier capping completed successfully!")
    return data
```

```
In [15]: # Numeric columns
numeric_cols = ['ProductPrice', 'Quantity', 'CustomerAge', 'ProductReviews']
```

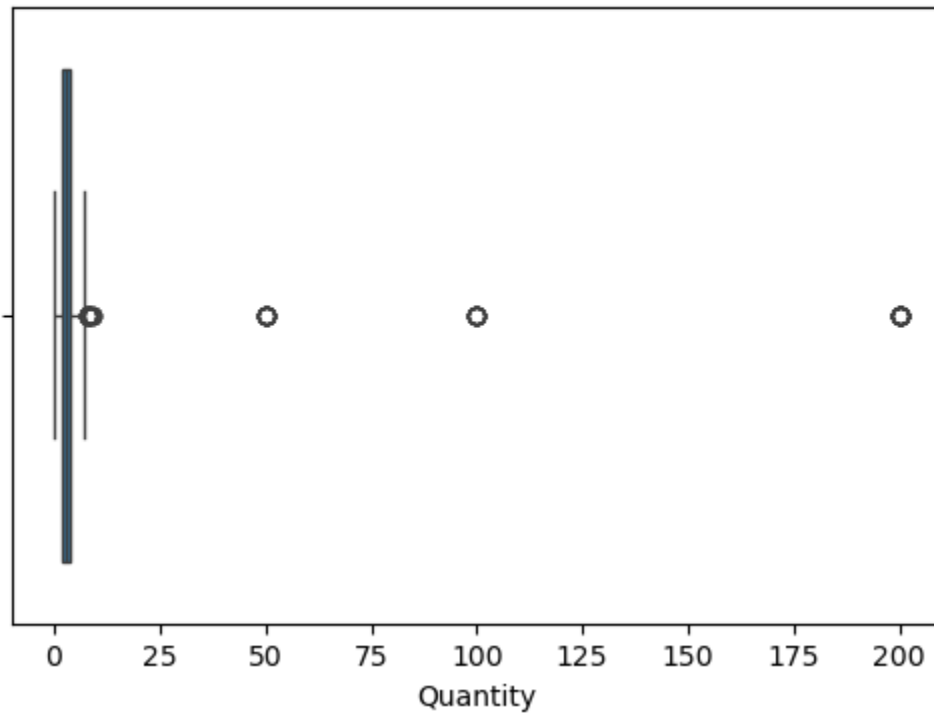
```
In [16]: # Boxplots BEFORE capping
print("Boxplots BEFORE Outlier Treatment")
for col in numeric_cols:
    plt.figure(figsize=(6, 4))
    sns.boxplot(x=data[col])
    plt.title(f"Before Outlier Treatment: {col}")
    plt.show()
```

Boxplots BEFORE Outlier Treatment

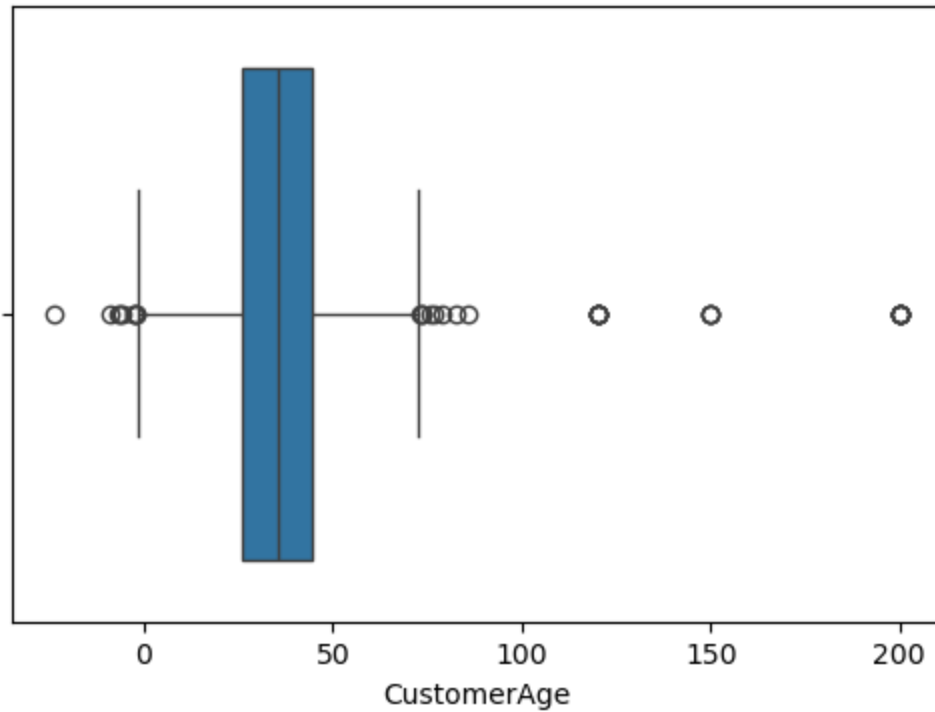
Before Outlier Treatment: ProductPrice



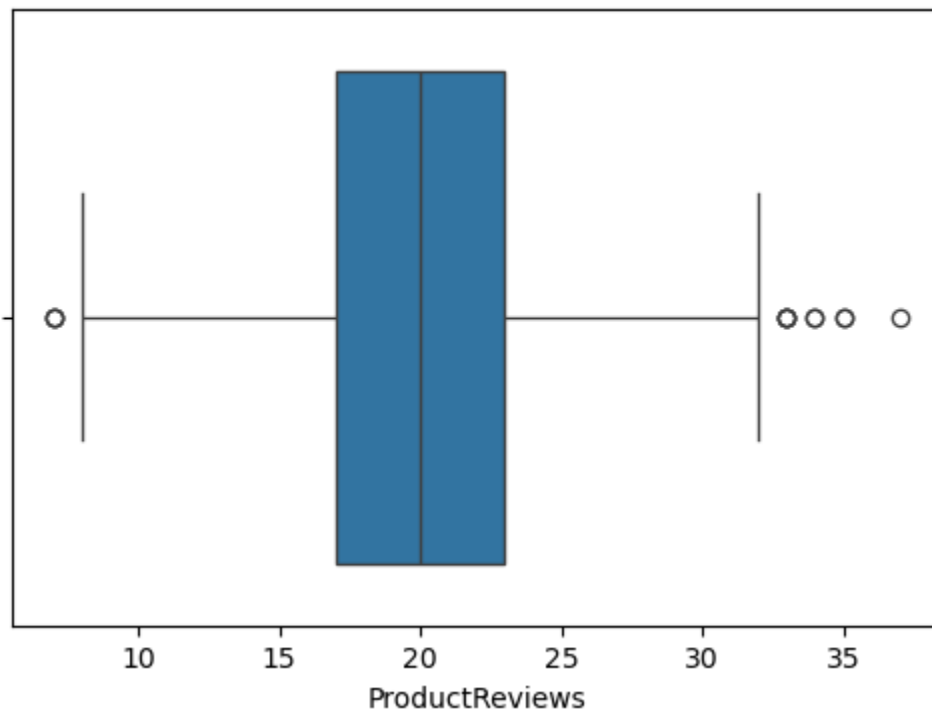
Before Outlier Treatment: Quantity



Before Outlier Treatment: CustomerAge



Before Outlier Treatment: ProductReviews



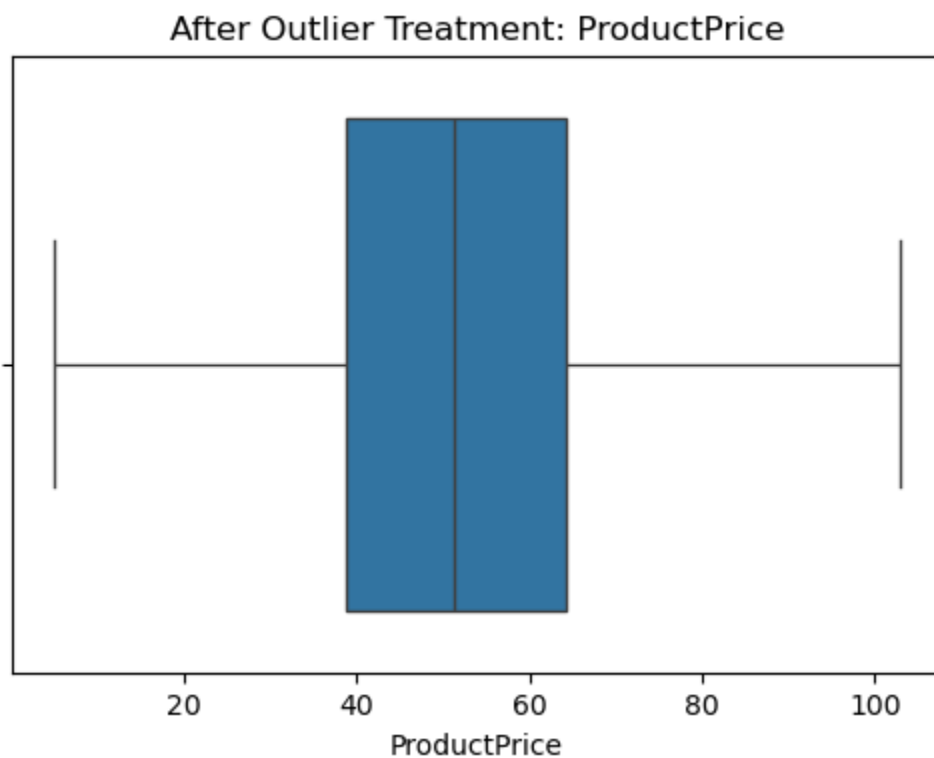
```
In [17]: # Apply Outlier Capping  
data = cap_outliers(data, numeric_cols)
```

ProductPrice: 56 outliers capped.
Quantity: 40 outliers capped.
CustomerAge: 37 outliers capped.
ProductReviews: 12 outliers capped.

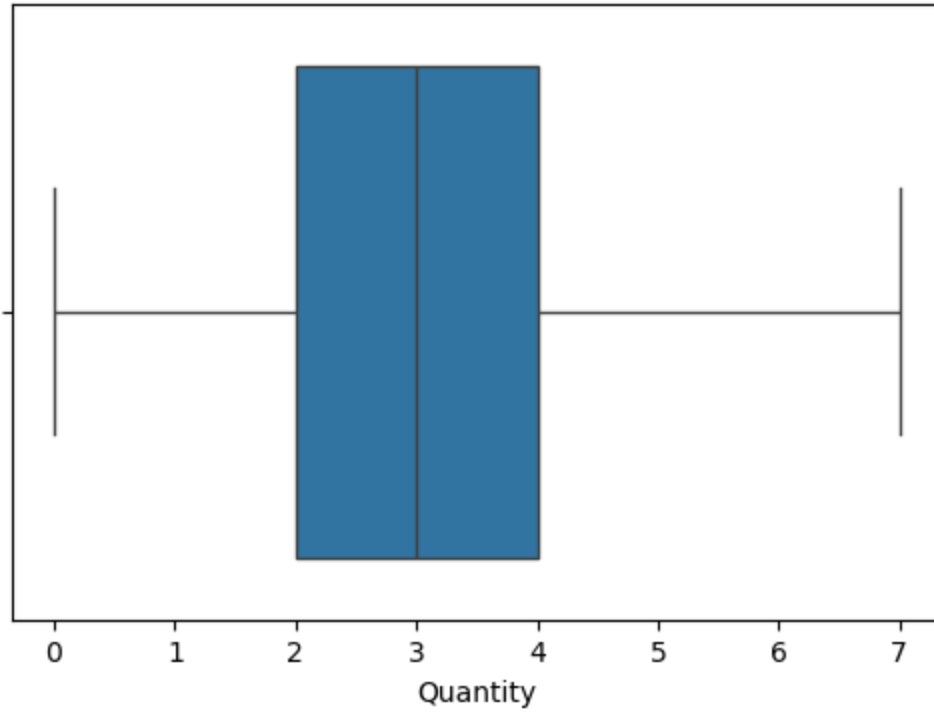
Outlier capping completed successfully!

```
In [18]: # Boxplots AFTER capping
print("\n Boxplots AFTER Outlier Treatment")
for col in numeric_cols:
    plt.figure(figsize=(6, 4))
    sns.boxplot(x=data[col])
    plt.title(f"After Outlier Treatment: {col}")
    plt.show()
```

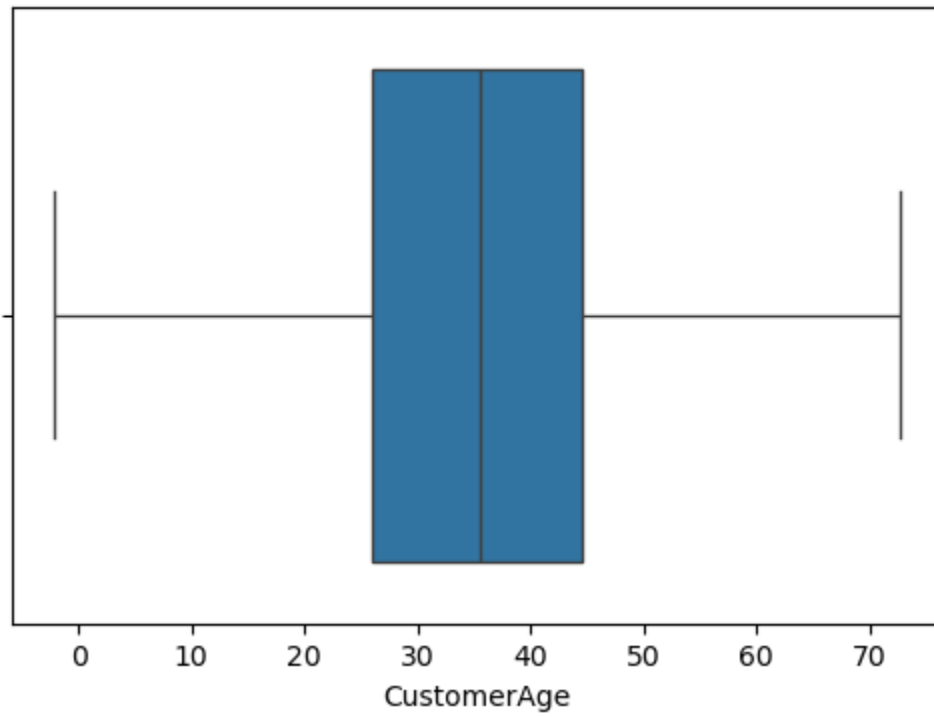
Boxplots AFTER Outlier Treatment

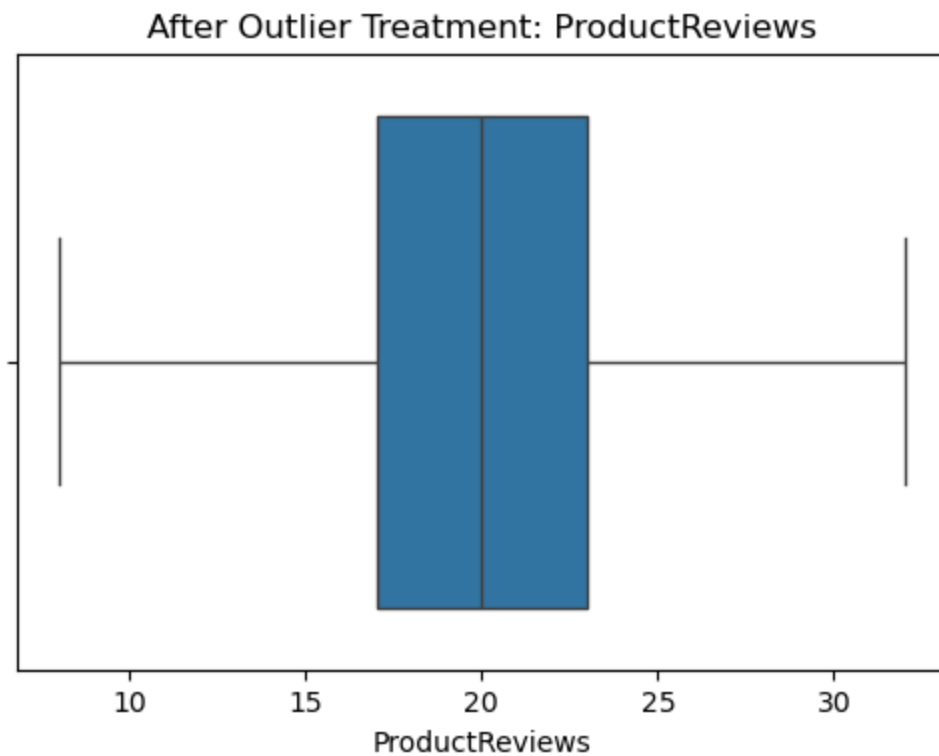


After Outlier Treatment: Quantity



After Outlier Treatment: CustomerAge





```
In [19]: # Apply Outlier Capping
data = cap_outliers(data, numeric_cols)
```

ProductPrice: 0 outliers capped.
Quantity: 0 outliers capped.
CustomerAge: 0 outliers capped.
ProductReviews: 0 outliers capped.

Outlier capping completed successfully!

Step 5: STANDARDIZE CATEGORICAL DATA

```
In [20]: # Remove extra whitespace and normalize case (Title Case)
data['DeliveryType'] = data['DeliveryType'].astype(str).str.strip().str.title()
data['ProductCategory'] = data['ProductCategory'].astype(str).str.strip().str.title()
data['PaymentMethod'] = data['PaymentMethod'].astype(str).str.strip().str.title()
```

```
In [21]: # Fix common spelling mistakes
data['ProductCategory'] = data['ProductCategory'].replace({
    'Eletronics': 'Electronics',
    'Book': 'Books',
    'Fasion': 'Fashion'
})
data['DeliveryType'] = data['DeliveryType'].replace({
    'Express ': 'Express',
    'Standard ': 'Standard'
})
```



```
})
```

```
In [22]: # Verify unique values
print("DeliveryType unique values:", data['DeliveryType'].unique())
print("ProductCategory unique values:", data['ProductCategory'].unique())
print("PaymentMethod unique values:", data['PaymentMethod'].unique())
```

DeliveryType unique values: ['Express' 'Standard' 'Overnight']

ProductCategory unique values: ['Clothing' 'Sports' 'Home' 'Electronics' 'Books']

PaymentMethod unique values: ['Paypal' 'Debit Card' 'Upi' 'Credit Card']

Step 6: Apply Transformation

```
In [23]: # Converting skewed to Normal Distribution
skew1=data['CustomerTenure'].skew()
print(skew1)
```

1.7849991498795355

```
In [24]: # Used Log Transformation
data['CustomerTenure']=np.log1p(data['CustomerTenure'])
```

```
In [25]: skew2=data['CustomerTenure'].skew()
print(skew2)
```

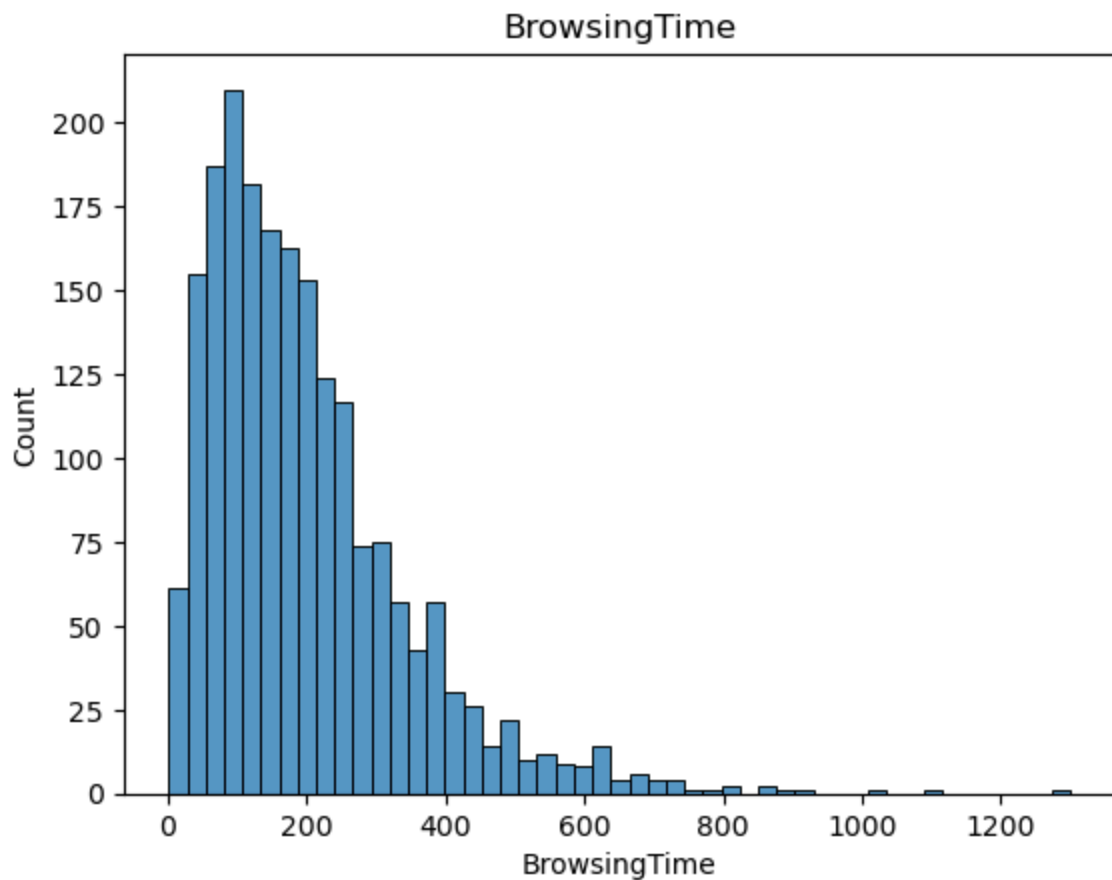
-0.45465701252384527

```
In [26]: # its normal distribution now
data['ProductReviews'].skew()
```

Out[26]: np.float64(0.2047011487568364)

```
In [27]: # Normal Distribution

#checking the distribution of BrowsingTime column
sns.histplot(x=data['BrowsingTime'])
plt.title('BrowsingTime')
plt.show()
```



```
In [28]: data['BrowsingTime'].skew()
```

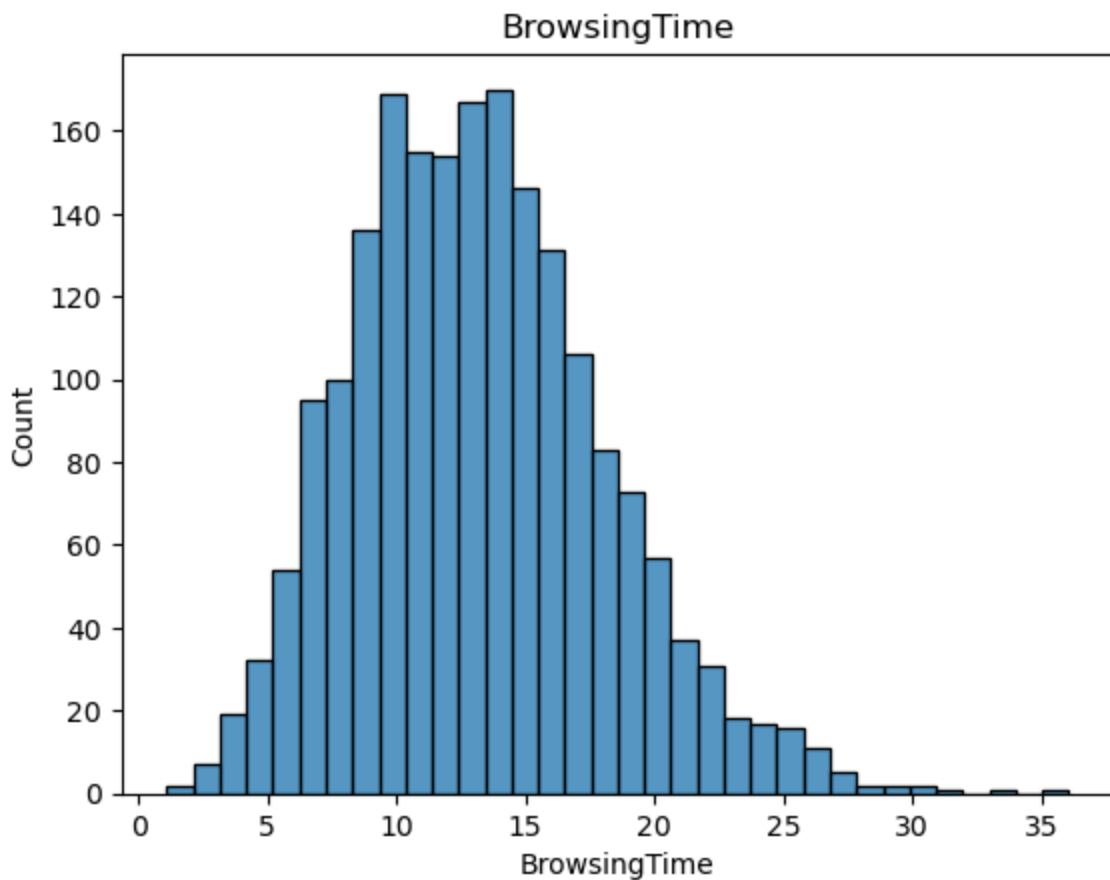
```
Out[28]: np.float64(1.678529302048084)
```

```
In [29]: # We have used sqrt transformation to convert into normal distribution  
data['BrowsingTime']=np.sqrt(data['BrowsingTime'])
```

```
In [30]: data['BrowsingTime'].skew()
```

```
Out[30]: np.float64(0.5248700518789231)
```

```
In [31]: #checking the distribution of BrowsingTime column after treating the outliers  
sns.histplot(x=data['BrowsingTime'])  
plt.title('BrowsingTime')  
plt.show()
```



Step 7: Binning

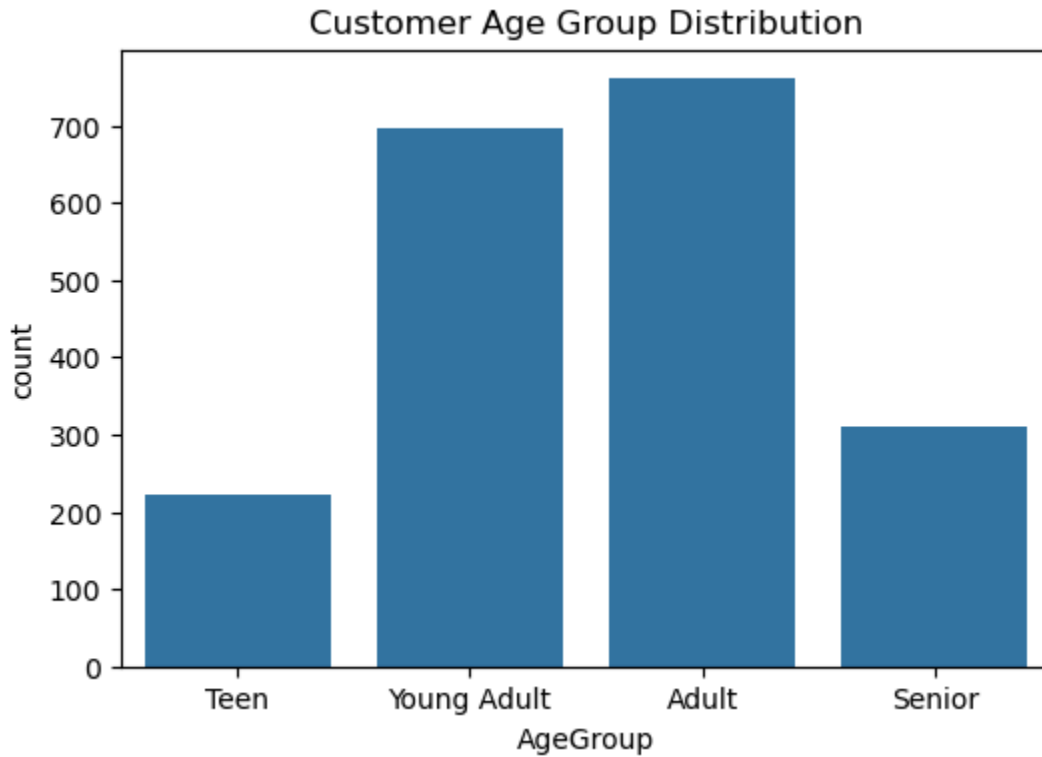
```
In [32]: # Create AgeGroup
data['AgeGroup'] = pd.cut(
    data['CustomerAge'],
    bins=[0, 18, 35, 50, 100],
    labels=['Teen', 'Young Adult', 'Adult', 'Senior'],
    include_lowest=True
)
```

```
In [33]: print("AgeGroup distribution:\n", data['AgeGroup'].value_counts())
```

```
AgeGroup distribution:
AgeGroup
Adult      760
Young Adult 695
Senior     309
Teen       222
Name: count, dtype: int64
```

```
In [34]: # Visualize Distributions
plt.figure(figsize=(6, 4))
sns.countplot(x="AgeGroup", data=data)
plt.title("Customer Age Group Distribution")
```

```
plt.show()
```



```
In [35]: print(data[['CustomerAge', 'AgeGroup']].isnull().sum())  
print("\nAgeGroup distribution:\n", data['AgeGroup'].value_counts())
```

```
CustomerAge    0  
AgeGroup      14  
dtype: int64
```

```
AgeGroup distribution:  
AgeGroup  
Adult      760  
Young Adult 695  
Senior     309  
Teen       222  
Name: count, dtype: int64
```

```
In [36]: data['AgeGroup'] = data['AgeGroup'].cat.add_categories(['Unknown'])  
data['AgeGroup'] = data['AgeGroup'].fillna('Unknown')
```

```
In [37]: print(data.isnull().sum())
```

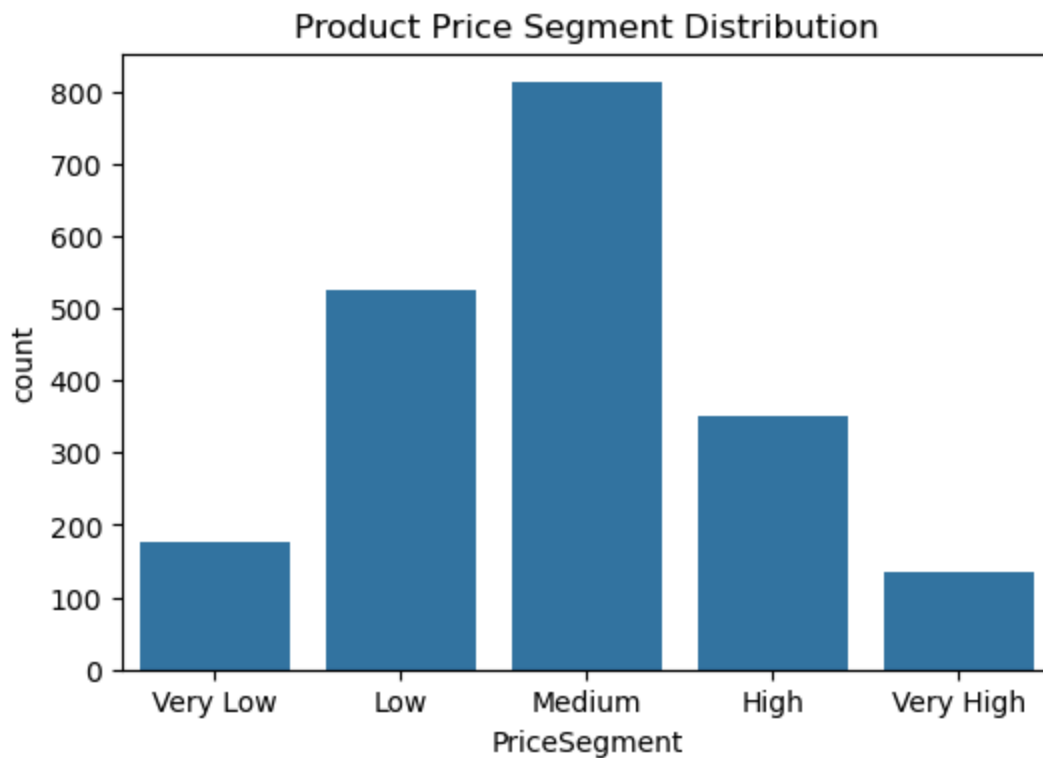
```
OrderID          0
ProductPrice     0
Quantity         0
CustomerAge      0
ProductReviews   0
ProductRating    0
DiscountPercentage 0
CustomerTenure   0
ProductCategory  0
DeliveryType     0
PaymentMethod    0
CustomerLocation 0
ReturnStatus     0
BrowsingTime     0
AgeGroup         0
dtype: int64
```

```
In [38]: # Create Price Segments (Fixed)
data['PriceSegment'] = pd.cut(
    data['ProductPrice'],
    bins=[5, 25, 45, 65, 85, 105],
    labels=['Very Low', 'Low', 'Medium', 'High', 'Very High'],
    include_lowest=True
)
```

```
In [39]: print("\nPriceSegment distribution:\n", data['PriceSegment'].value_counts())
```

```
PriceSegment distribution:
PriceSegment
Medium      812
Low         526
High        350
Very Low    176
Very High   136
Name: count, dtype: int64
```

```
In [40]: plt.figure(figsize=(6, 4))
sns.countplot(x="PriceSegment", data=data)
plt.title("Product Price Segment Distribution")
plt.show()
```

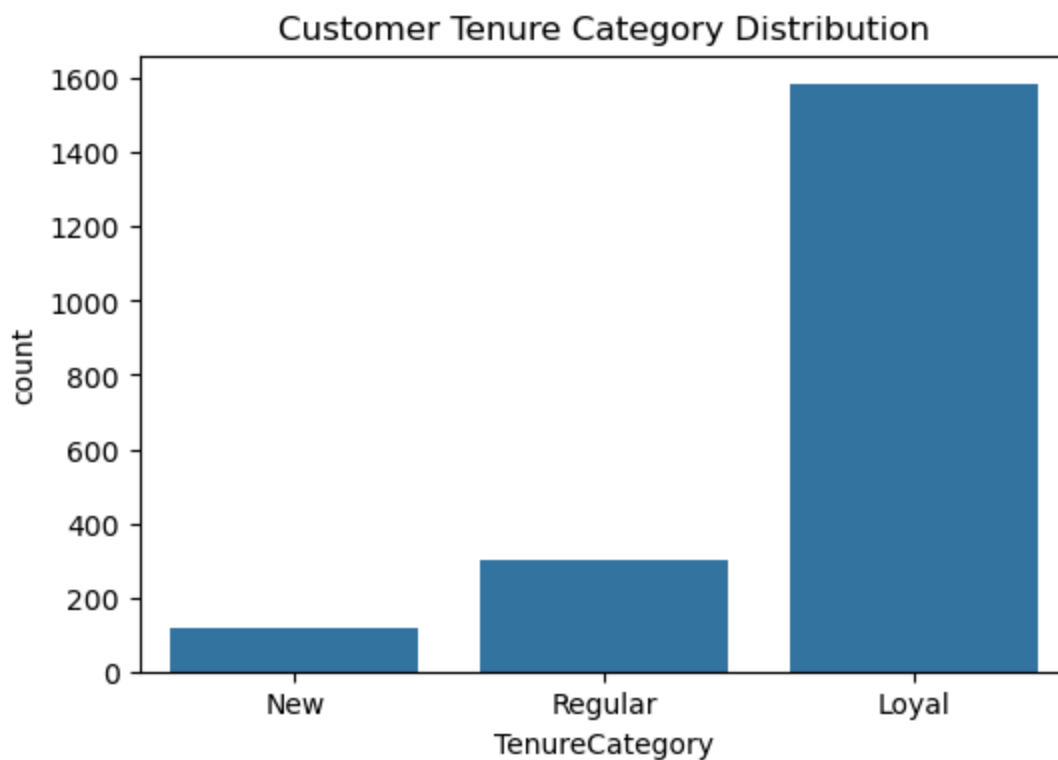


```
In [41]: # Create Tenure Categories (3 intervals)
data["TenureCategory"] = pd.cut(
    data["CustomerTenure"],
    bins=[0, 1, 2, np.inf],
    labels=['New', 'Regular', 'Loyal'],
    include_lowest=True
)
```

```
In [42]: print("\nTenureCategory distribution:\n", data['TenureCategory'].value_counts())

TenureCategory distribution:
TenureCategory
Loyal      1583
Regular    302
New        115
Name: count, dtype: int64
```

```
In [43]: plt.figure(figsize=(6, 4))
sns.countplot(x="TenureCategory", data=data)
plt.title("Customer Tenure Category Distribution")
plt.show()
```



```
In [44]: print(data.isnull().sum())
```

```
OrderID          0
ProductPrice     0
Quantity         0
CustomerAge      0
ProductReviews   0
ProductRating    0
DiscountPercentage 0
CustomerTenure   0
ProductCategory  0
DeliveryType     0
PaymentMethod    0
CustomerLocation 0
ReturnStatus     0
BrowsingTime     0
AgeGroup         0
PriceSegment     0
TenureCategory   0
dtype: int64
```

Step 8: Encode Categorical Variables

```
In [45]: # One-Hot Encoding for selected nominal columns
one_hot_cols = ['DeliveryType', 'PaymentMethod', 'CustomerLocation', 'ReturnStatus']
data_onehot = pd.get_dummies(data, columns=one_hot_cols, dtype=int)
```

```
In [46]: # Verify the new DataFrame
print("Columns after one-hot encoding:\n", data_onehot.columns)
print("\nShape of DataFrame before encoding:", data.shape)
print("Shape of DataFrame after encoding:", data_onehot.shape)
```

Columns after one-hot encoding:

```
Index(['OrderID', 'ProductPrice', 'Quantity', 'CustomerAge', 'ProductReviews',
      'ProductRating', 'DiscountPercentage', 'CustomerTenure',
      'ProductCategory', 'BrowsingTime', 'AgeGroup', 'PriceSegment',
      'TenureCategory', 'DeliveryType_Express', 'DeliveryType_Overnight',
      'DeliveryType_Standard', 'PaymentMethod_Credit Card',
      'PaymentMethod_Debit Card', 'PaymentMethod_Paypal', 'PaymentMethod_Upi',
      'CustomerLocation_Central', 'CustomerLocation_East',
      'CustomerLocation_North', 'CustomerLocation_South',
      'CustomerLocation_West', 'ReturnStatus_No', 'ReturnStatus_Yes'],
      dtype='object')
```

Shape of DataFrame before encoding: (2000, 17)

Shape of DataFrame after encoding: (2000, 27)

Step 9: Feature Engineering

```
In [47]: # Interaction Features
data['price_rating_interaction'] = data['ProductPrice'] * data['ProductRating']
data['quantity_reviews_interaction'] = data['Quantity'] * data['ProductReviews']
```

```
In [48]: # Derived Features
data['discounted_price'] = data['ProductPrice'] * (1 - data['DiscountPercentage'])
data['price_per_review'] = data['ProductPrice'] / (data['ProductReviews'] + 1)
```

```
In [49]: data
```


Out[49]:

	OrderID	ProductPrice	Quantity	CustomerAge	ProductReviews	ProductI
0	1	102.87625	1.0	44.184047	18.0	
1	2	47.23000	1.0	11.608463	11.0	
2	3	62.95000	4.0	18.834189	26.0	
3	4	80.46000	1.0	40.147378	20.0	
4	5	45.32000	3.0	44.344160	19.0	
...	...	...	...	...	...	
1995	1996	71.40000	5.0	38.211665	18.0	
1996	1997	49.47000	2.0	42.336958	17.0	
1997	1998	32.36000	5.0	35.827560	21.0	
1998	1999	46.74000	1.0	34.990873	18.0	
1999	2000	35.10000	2.0	52.949407	15.0	

2000 rows × 21 columns

In [50]: *# Create TotalSpend first*
`data['TotalSpend'] = data['ProductPrice'] * data['Quantity']`

In [51]: *# Indicator Variables*
`data['high_value_order'] = (data['TotalSpend'] > 700).astype(int)`

In [52]: *# Frequent buyer (CustomerTenure > 36 months)*
`data['frequent_buyer'] = (data['CustomerTenure'] > 36).astype(int)`

In [53]: *# Adult customer flag*
`data['is_adult'] = (data['AgeGroup'] == 'Adult').astype(int)`

In [54]: *# Verify*
`print(data[['ProductPrice', 'Quantity', 'TotalSpend', 'high_value_order', 'Cus`

	ProductPrice	Quantity	TotalSpend	high_value_order	CustomerTenure \
0	102.87625	1.0	102.87625	0	2.823757
1	47.23000	1.0	47.23000	0	2.188296
2	62.95000	4.0	251.80000	0	3.441699
3	80.46000	1.0	80.46000	0	3.972177
4	45.32000	3.0	135.96000	0	0.009950

	AgeGroup
0	Adult
1	Teen
2	Young Adult
3	Adult
4	Adult

```
In [55]: # Document new features
new_features = [
    'price_rating_interaction',
    'quantity_reviews_interaction',
    'discounted_price',
    'TotalSpend',
    'high_value_order',
    'frequent_buyer',
    'is_adult'
]
print("New features added:", new_features)
```

New features added: ['price_rating_interaction', 'quantity_reviews_interaction', 'discounted_price', 'TotalSpend', 'high_value_order', 'frequent_buyer', 'is_adult']

Step 10: Save & Validate

```
In [63]: # Save the cleaned dataset
output_path = r"C:\Users\li\Desktop\Arbor Academy Project\ecommerce_dataset.xlsx"
data.to_excel(output_path, index=False)
print(f"Cleaned dataset saved to: {output_path}")
```

Cleaned dataset saved to: C:\Users\li\Desktop\Arbor Academy Project\ecommerce_dataset.xlsx

```
In [64]: # Compare before/after
print("\n Dataset Shape")
print("Before cleaning:", (2010, 15)) # replace with actual before if tracked
print("After cleaning:", data.shape)
```

Dataset Shape
Before cleaning: (2010, 15)
After cleaning: (2000, 25)

```
In [65]: # Example: Check missing values
print("\nMissing values after cleaning:\n", data.isnull().sum())
```

```
Missing values after cleaning:
  OrderID          0
  ProductPrice     0
  Quantity         0
  CustomerAge      0
  ProductReviews   0
  ProductRating    0
  DiscountPercentage 0
  CustomerTenure   0
  ProductCategory  0
  DeliveryType     0
  PaymentMethod    0
  CustomerLocation 0
  ReturnStatus     0
  BrowsingTime     0
  AgeGroup         0
  PriceSegment     0
  TenureCategory   0
  price_rating_interaction 0
  quantity_reviews_interaction 0
  discounted_price 0
  price_per_review 0
  TotalSpend       0
  high_value_order 0
  frequent_buyer   0
  is_adult         0
dtype: int64
```

```
In [395... # Generate summary report
# Numeric summary
num_summary = data.describe().T
num_summary['missing_values'] = data.isnull().sum()
num_summary['unique_values'] = data.nunique()
print("\nNumeric Summary:\n", num_summary)

# Categorical summary
cat_summary = data.describe(include=['object', 'category']).T
cat_summary['missing_values'] = data.isnull().sum()
cat_summary['unique_values'] = data.nunique()
print("\nCategorical Summary:\n", cat_summary)
```

Numeric Summary:

	count	mean	std	min \
OrderID	2000.0	1000.500000	577.494589	1.000000
ProductPrice	2000.0	52.298525	20.528305	5.000000
Quantity	2000.0	2.989000	1.628252	0.000000
CustomerAge	2000.0	35.437228	14.604630	-2.149116
ProductReviews	2000.0	19.985500	4.312608	8.000000
ProductRating	2000.0	3.049090	1.154147	1.000000
DiscountPercentage	2000.0	25.345055	14.588209	0.120000
CustomerTenure	2000.0	2.824203	1.026089	0.000000
BrowsingTime	2000.0	13.204965	4.884180	1.113553
price_rating_interaction	2000.0	159.164023	88.994543	5.050000
quantity_reviews_interaction	2000.0	59.619500	35.530310	0.000000
discounted_price	2000.0	39.061259	17.341024	2.644000
price_per_review	2000.0	2.606853	1.209389	0.166667
TotalSpend	2000.0	155.935637	107.357478	0.000000
high_value_order	2000.0	0.000500	0.022361	0.000000
frequent_buyer	2000.0	0.000000	0.000000	0.000000
is_adult	2000.0	0.380000	0.485508	0.000000

	25%	50%	75% \
OrderID	500.750000	1000.500000	1500.250000
ProductPrice	38.707500	51.400000	64.375000
Quantity	2.000000	3.000000	4.000000
CustomerAge	25.890303	35.491605	44.583248
ProductReviews	17.000000	20.000000	23.000000
ProductRating	2.080000	3.045000	4.010000
DiscountPercentage	12.807500	25.740000	38.005000
CustomerTenure	2.172192	2.919391	3.587469
BrowsingTime	9.709531	12.866623	16.189039
price_rating_interaction	90.790000	145.078350	213.029150
quantity_reviews_interaction	34.000000	54.000000	80.000000
discounted_price	26.868898	36.972745	49.201951
price_per_review	1.772414	2.447619	3.230891
TotalSpend	73.677500	139.910000	210.952500
high_value_order	0.000000	0.000000	0.000000
frequent_buyer	0.000000	0.000000	0.000000
is_adult	0.000000	0.000000	1.000000

	max	missing_values	unique_values
OrderID	2000.000000	0	2000
ProductPrice	102.876250	0	1596
Quantity	7.000000	0	8
CustomerAge	72.622667	0	1866
ProductReviews	32.000000	0	25
ProductRating	5.000000	0	401
DiscountPercentage	50.000000	0	1670
CustomerTenure	5.089262	0	1683
BrowsingTime	36.064526	0	1957
price_rating_interaction	485.575900	0	1975
quantity_reviews_interaction	210.000000	0	107
discounted_price	102.207554	0	1997
price_per_review	9.675556	0	1850
TotalSpend	720.133750	0	1697

high_value_order	1.000000	0	2
frequent_buyer	0.000000	0	1
is_adult	1.000000	0	2

Categorical Summary:

	count	unique	top	freq	missing_values	\
ProductCategory	2000	5	Clothing	413	0	
DeliveryType	2000	3	Overnight	710	0	
PaymentMethod	2000	4	Credit Card	529	0	
CustomerLocation	2000	5	West	440	0	
ReturnStatus	2000	2	Yes	1014	0	
AgeGroup	2000	5	Adult	760	0	
PriceSegment	2000	5	Medium	812	0	
TenureCategory	2000	3	Loyal	1583	0	

	unique_values
ProductCategory	5
DeliveryType	3
PaymentMethod	4
CustomerLocation	5
ReturnStatus	2
AgeGroup	5
PriceSegment	5
TenureCategory	3

```
In [66]: import os
print(os.getcwd())
```

C:\Users\li

```
In [67]: # replace df with the actual variable name (e.g. data, df_cleaned)
output_path = r"C:\Users\li\Desktop\Arbor Academy Project\ecommerce_dataset.xlsx"
data.to_excel(output_path, index=False)
```

In []: